INTRODUZIONE A VUE.JS



FRAMEWORK (JAVASCRIPT)

cos'è un framework

- più che librerie
- inversione di controllo
- estensibile
- non modificabile dall'utente (a differenza dei template)

Attacchiamo la nostra logica ad un sistema già funzionante



PERCHÉ I FRAMEWORK?

- javascript non è stato progettato per lo sviluppo di applicazioni
- anche se js è in continua estensione, alcune cose non possono farne parte
- molto "codice boilerplate"



FRAMEWORK JAVASCRIPT COMUNI

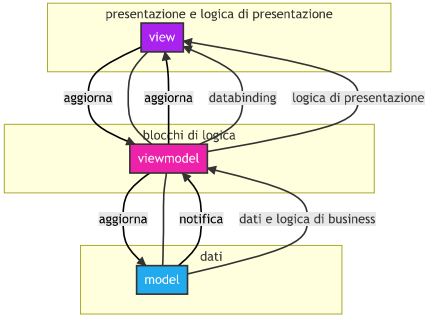
- react
- angular
- vue.js
- ember.js
- gwt
- (molti altri, incluso quello inventato pochi minuti fa)



FRAMEWORK WEB: MODEL-VIEW-BINDER

- view: presentazione e logica di presentazione
- viewmodel: databinding
- model: logica di business e dati



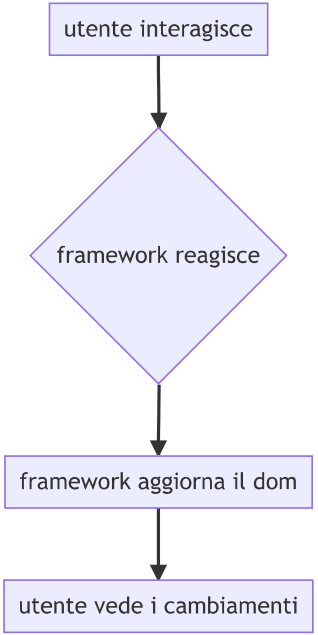

presentazione e logica di presentazione
view
aggiorna
aggiorna
databinding
logica di presentazione
blocchi di logica
viewmodel
aggiorna
notifica
dati e logica di business
dati
model

TYPESCRIPT

alcuni framework javascript supportano typescript, un superset sintattico di javascript.
non useremo typescript, ma potresti volerlo esaminare se ti occuperai di svilupp front-end.



COME FUNZIONA LO SVILUPPO FRONT-END?

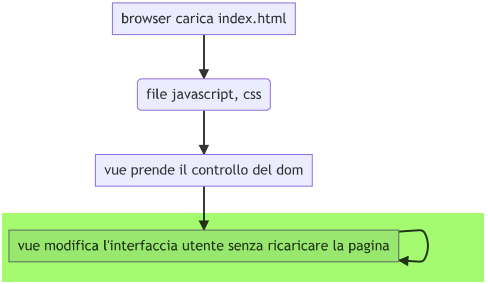


COS'È VUE.JS

- framework progressivo per la creazione di interfacce utente.
- "progressivo" significa che è progettato per essere adottato in modo incrementale.
- è facile integrare vue con altri progetti o librerie.
- può essere utilizzato come alternativa a jquery o in progetti single page application (spa).



APPROCCIO SPA (SINGLE PAGE APPLICATION)



VUE.JS PLAYGROUND

esiste un ambiente ufficiale per testare codice vue direttamente nel browser (utile per prototipi veloci o per condividere riproduzioni di bug):

- https://sfc.vuejs.org/



FUNZIONALITÀ PRINCIPALI

funzionalità core:

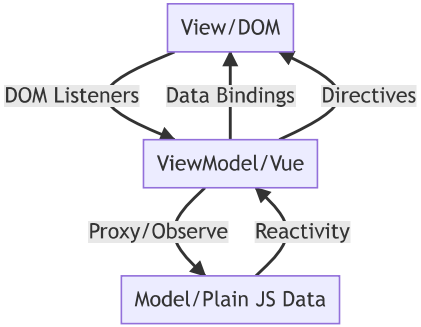
- **rendering dichiarativo**: estende l'html standard con una sintassi di template che permette di descrivere dichiarativamente l'output html in base allo stato javascript.
- **reattività**: il sistema traccia automaticamente le modifiche allo stato javascript e aggiorna il dom in modo efficiente quando cambiano.



MODI DI UTILIZZARE VUE.JS

- **standalone**: usato come una libreria (simile a jquery) importata in una pagina.
- **embedded web components**: incorporare vue.js in applicazioni web legacy esistenti.
- **spa (single-page application)**:
  - viene eseguito interamente sul client.
- **full-stack / server-side rendering (ssr)**:
  - viene eseguito sia sul server che sul client (es. nuxt.js).
- **static site generation (ssg)**:
  - compila un sito web statico.
  - nessun codice vue in esecuzione su server/client (solo html generato).

MODEL-VIEW-BINDER (DIAGRAMMA)



Useremo questo stile

STILE API: OPTIONS VS COMPOSITION

vue offre due stili possibili per scrivere i componenti:

- **options api**: definisce la logica del componente usando un oggetto con opzioni come `data`, `methods`, `mounted`. è considerato più amichevole per i principianti e per chi viene da vue 2.
- **composition api**: definisce la logica importando funzioni api. offre maggiore flessibilità e migliore riutilizzo del codice.



ESEMPIO MINIMO (PARTE JS - OPTIONS API)

questo esempio mostra l'approccio classico (options api) senza un sistema di build.

```
import { createApp } from 'vue'

createApp({
  // la funzione data restituisce lo stato reattivo
  data() {
    return {
      count: 0
    }
  }
}).mount('#app')
```



ESEMPIO MINIMO (TEMPLATE)

il template html corrispondente all'esempio js.

```
<div id="app">
  <!-- ascolta l'evento click e incrementa count -->
  <button @click="count++">
    Count is: {{ count }}
  </button>
</div>
```



TEMPLATE DI COMPONENTI VUE

i componenti vue hanno un'estensione `.vue` e solitamente contengono tre blocchi:

- `<template>`: contiene il markup html (con sintassi speciale vue).
- `<script>`: contiene la logica javascript del componente.
- `<style>`: contiene il css specifico per il componente.



SFC

Un componente a file singolo (SFC, Single-File Component) è un file simile all'HTML che contiene:

- JavaScript
- HTML (template)
- CSS

Un SFC consente di riutilizzare i componenti.

Esempio di SFC: estendere il pulsante HTML standard con un comportamento personalizzato.



```
<script>
export default {
data() {
return { count: 0 }
}
}
</script>
<template>
<button @click="count++">Count is: {{ count }}</button>
</template>
<style scoped>
button { font-weight: bold; }
</style>
```



```
<script>
export default {
data() {
return { count: 0 }
}
}
</script>
<template>
<button @click="count++">Count is: {{ count }}</button>
</template>
<style scoped>
button { font-weight: bold; }
</style>
```



PAGINA INDEX

`index.html` contiene l'html per la single page. può contenere codice non-vue, come js/css esterni o custom.

esempio:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Example app</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <div id="app"></div>
    <!-- il punto di ingresso per i moduli javascript -->
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

MAIN JS

il file `main.js` è il punto di ingresso dell'applicazione. crea e monta l'app.

```
import {createApp, reactive} from 'vue'
import router from './router'
import App from './App.vue'
import Component1 from './components/Component1.vue'
const app = createApp(App)
app.component("Component1", Component1);
app.use(router)
app.mount('#app')
```

APP.VUE

in `app.vue` abbiamo il componente principale della spa.

```
<script setup>
import { RouterView } from 'vue-router'
</script>
<script>
export default {}
</script>

<template>
    <header>App header</header>
    <main><RouterView ></RouterView></main>
</template>

<style>
</style>
```

ROUTER

il router gestisce le rotte per le viste all'interno dell'app.

```
import {createRouter, createWebHashHistory} from 'vue-router'
import HomeView from '../views/HomeView.vue'
import Page1View from '../views/Page1View.vue'
import Page2View from '../views/Page2View.vue'
const router = createRouter({
    history: createWebHashHistory(),
    routes: [
        {path: '/', component: HomeView},
        {path: '/link1', component: Page1View},
        {path: '/some/:id/link', component: Page2View},
    ]
});
export default router
```

ROUTER

Il codice precedente crea un "router" che è possibile gestire modificando gli URL
Ciascun URL è associato a una diversa vista dell'applicazione. L'effettivo link pe
`/link1` sarà qualcosa come `https://example.com/#/link1`.



INSTALLAZIONE

usiamo vue con **vite**. vite è un sistema di build di prossima generazione per lo sviluppo web.

```
npm create vue@latest
```



APPROFONDIMENTI

VIEW (LA VISTA)

- **cos'è**: è la rappresentazione visiva dell'applicazione (il dom, html + css).
- **responsabilità**:
  - è passiva: non prende decisioni.
  - si occupa solo di *come* mostrare i dati all'utente.
  - intercetta le interazioni (click, input) e le passa al viewmodel.
- **in vue.js**: corrisponde alla sezione `<template>`.



VIEWMODEL (IL MEDIATORE)

- **cos'è**: un oggetto che rappresenta lo stato della vista e la logica di presentazione.
- **responsabilità**:
  - agisce da ponte tra il **model** (dati grezzi) e la **view**.
  - espone dati "pronti per la visualizzazione" (reattività).
  - gestisce i comandi dell'utente e aggiorna i dati.
- **in vue.js**: corrisponde alla sezione `<script>` (variabili reattive `ref`, funzioni).

JAVASCRIPT: ORIGINI DI SCRIPTING

- **"javascript non è stato progettato per lo sviluppo di applicazioni"**
  - **storia**: creato in soli 10 giorni nel 1995 per netscape navigator.
  - **scopo originale**: aggiungere piccole interattività (validazione form, alert) a documenti statici, non costruire software complessi (come gmail o spotify).
  - **mancanze storiche**:
    - assenza di un sistema di moduli nativo (fino a es6).
    - gestione dello scope globale problematica (variabili che si sovrascrivono).
    - tipizzazione debole che scala male su grandi codebase.

LIMITI DELL'EVOLUZIONE DI JS

- **"anche se js è in continua estensione, alcune cose non possono farne parte"**
  - **limiti architetturali**: lo standard (ecmascript) definisce la sintassi, non l'architettura software.
  - **cosa manca nel linguaggio standard**:
    - **state management**: non esiste un modo nativo per sincronizzare automaticamente variabili js e dom (reattività).
    - **virtual dom**: manipolare il dom reale è lento; i framework aggiungono uno strato di astrazione per le performance.
    - **component model**: sebbene esistano i web components, i framework offrono un sistema di componenti molto più integrato e potente.

